Assignment 1: Adaline and Logistic Regression

Joshua Sadler

Febuary 13, 2026

Purpose

- Understand Adaline and Logistic Regression algorithms.
- Investigate performance differences between the two algorithms.
- Understand linear classification.
- Understand Stochastic Gradient Descent (SGD) and mini-batch SGD.

1 Task 1: Bias Absorption into Weight Vector

1.1 Original Model Formulation

The original Adaline and Logistic Regression models use a bias term b:

$$z = \mathbf{w}^T \mathbf{x} + b \tag{1}$$

where:

- $\mathbf{w} \in \mathbb{R}^d$ is the weight vector
- $\mathbf{x} \in \mathbb{R}^d$ is the feature vector
- $b \in \mathbb{R}$ is the bias

1.2 Bias Absorption

I use an augmented input vector:

$$\tilde{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} \tag{2}$$

and an augmented weight vector:

$$\tilde{\mathbf{w}} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \tag{3}$$

The model becomes:

$$z = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}} \tag{4}$$

1.3 Why the Models Are Equivalent

Since:

$$\tilde{\mathbf{w}}^T \tilde{\mathbf{x}} = b \cdot 1 + \mathbf{w}^T \mathbf{x}$$

the absorbed-bias model produces the same decision function as the original formulation. Therefore, both models are mathematically equivalent.

The only difference is implementation convenience: eliminate the separate bias update and instead update all weights uniformly.

Implementation Notes

- Added a column of ones to the input matrix.
- Increased weight vector dimension by 1.
- Removed explicit bias parameter.

2 Task 2: Convergence Comparison on Iris and Wine Datasets

2.1 Experimental Setup

Binary classification tasks were constructed as follows:

- **Iris dataset:** Setosa vs Versicolor
- **Wine dataset:** Class 1 vs Class 2

2.2 Loss Functions Used

Adaline minimizes Mean Squared Error (MSE):

$$L(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{5}$$

Logistic Regression minimizes Log Loss:

$$L(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)] \tag{6}$$

2.3 Results on Iris Dataset

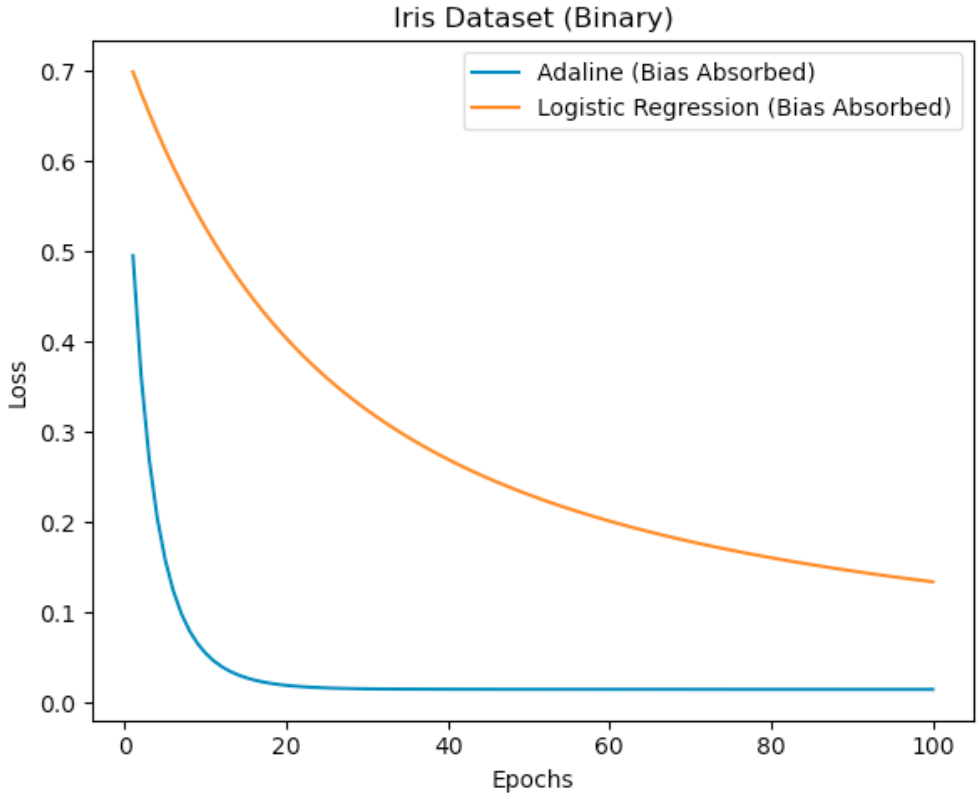

Figure 1: Loss convergence comparison on Iris dataset (binary).

Observation:

- Adaline converges rapidly due to quadratic loss structure.
- Logistic regression converges more gradually due to probabilistic modeling.

2.4 Results on Wine Dataset

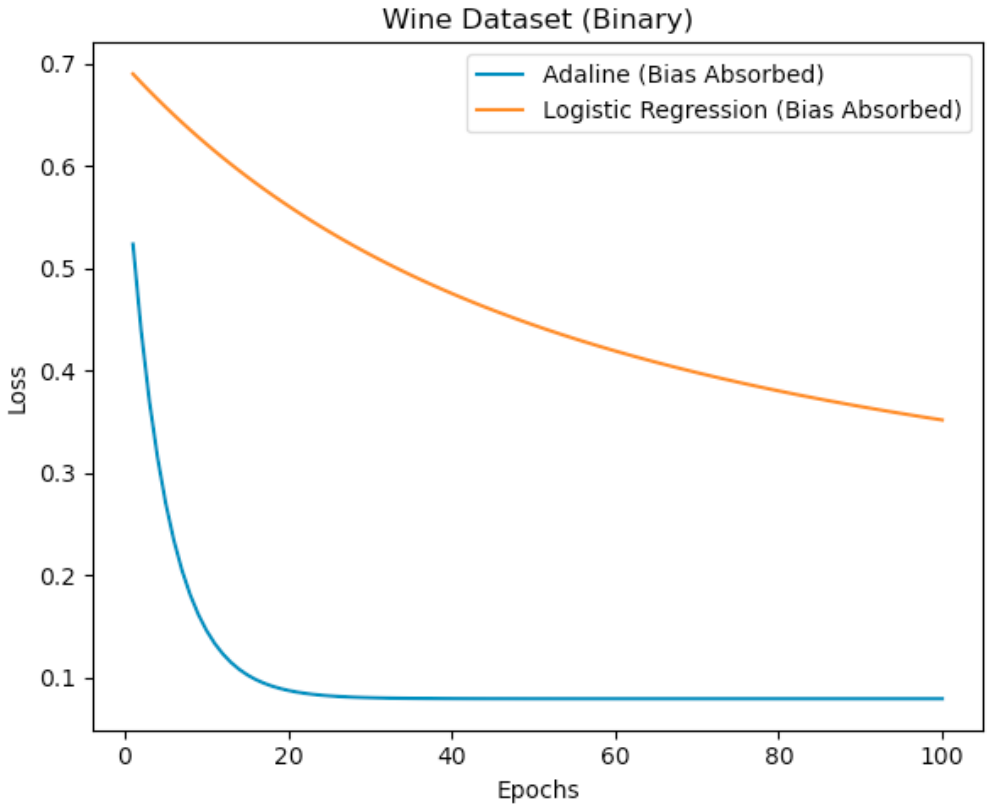

Figure 2: Loss convergence comparison on Wine dataset (binary).

Observation:

- Adaline again shows rapid MSE decay.
- Logistic regression converges more slowly and exhibits a different curvature.

2.5 Analysis

The differences arise from the loss functions:

- MSE is symmetric and quadratic.
- Log-loss penalizes confident misclassifications more heavily.

Since the Wine dataset has different feature distribution and class separation than Iris, logistic regression exhibits noticeably different convergence behavior, while Adaline behaves similarly due to the quadratic nature of MSE.

3 Task 3: Multiclass Classification Using Perceptrons

The perceptron algorithm is inherently a binary classifier, meaning it can only distinguish between two classes. However, the Iris dataset contains three classes: *setosa*, *versicolor*, and *virginica*. Since I was only allowed to use perceptrons (with no limit on the number of models), I implemented a multiclass classification strategy using multiple binary perceptrons.

3.1 Method: One-vs-Rest (OvR) Strategy

To perform multiclass classification, I implemented the **One-vs-Rest (OvR)** approach.

For each class k, I constructed a binary classification problem defined as:

$$y^{(k)} = \begin{cases} 1 & \text{if the sample belongs to class } k \\ 0 & \text{otherwise} \end{cases}$$

Thus, for the three classes in the Iris dataset, I trained three separate perceptrons:

- Perceptron 0: Setosa vs. (Versicolor + Virginica)
- Perceptron 1: Versicolor vs. (Setosa + Virginica)
- Perceptron 2: Virginica vs. (Setosa + Versicolor)

Each perceptron learns a linear decision boundary of the form:

$$g_k(x) = w_k^T x + b_k$$

where w_k and b_k are the learned weights and bias corresponding to class k.

3.2 Prediction Rule

During prediction, each perceptron produces a linear activation score:

$$g_k(x) = w_k^T x + b_k$$

I collect these activation values into a vector:

$$\mathbf{g}(x) = [g_0(x), g_1(x), g_2(x)]$$

The final predicted class is determined using the argmax rule:

$$\hat{y} = \arg\max_k g_k(x)$$

This selects the class whose perceptron produces the largest activation value. Geometrically, this corresponds to choosing the hyperplane for which the input sample lies farthest on the positive side.

3.3 Correctness of the Method

The OvR method works for the following reasons:

1. Each perceptron learns a separating hyperplane distinguishing one class from the remaining classes.
2. The activation value $g_k(x)$ represents the signed distance from the decision boundary.
3. Selecting the maximum activation effectively chooses the classifier that is most confident about its prediction.

Since the Iris dataset (using petal length and petal width) is nearly linearly separable—especially for the *setosa* class—the OvR approach performs well.

After standardizing the features and shuffling the dataset to prevent bias caused by ordered samples, the OvR perceptron achieved:

$$\textbf{Accuracy} \approx 0.91$$

This demonstrates that multiple binary perceptrons can be combined to perform effective multiclass classification.

3.4 Visualization of Decision Regions

Figure 3 shows the learned decision regions for the three classes using the OvR perceptron approach.

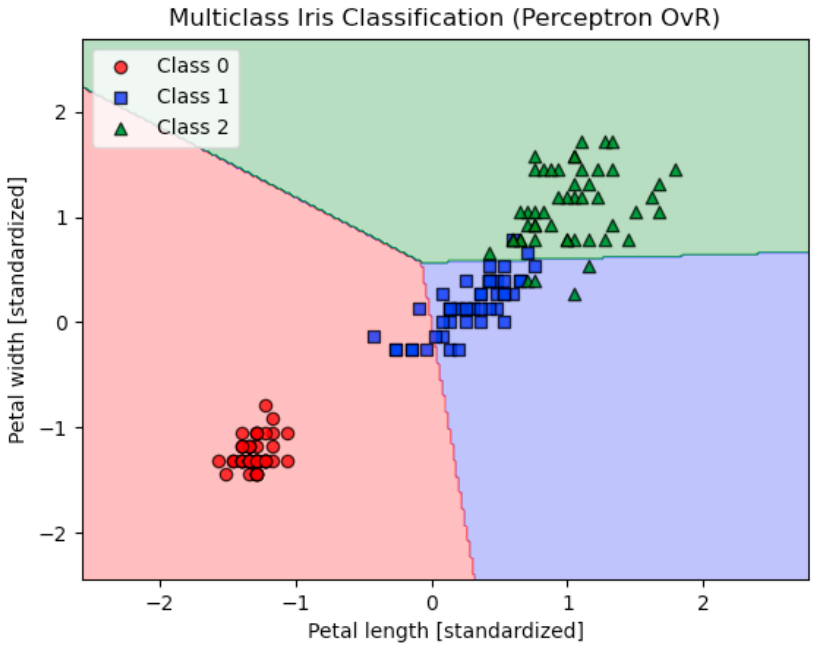

Figure 3: Multiclass classification of the Iris dataset using the One-vs-Rest perceptron method.

Task 4: Comparison of GD, SGD, and Mini-Batch SGD

In this task, I implemented three optimization strategies for logistic regression:

- Full Gradient Descent (GD)
- Stochastic Gradient Descent (SGD)
- Mini-Batch Gradient Descent

All three methods were evaluated on the binary Wine dataset (classes 1 and 2). To ensure a fair comparison, I used the same hyperparameters (learning rate η and number of epochs) for all methods.

Loss Function

For logistic regression, the loss function is the average cross-entropy loss:

$$L(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

where

$$\hat{y}^{(i)} = \sigma(\mathbf{w}^T\mathbf{x}^{(i)})$$

and

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

is the sigmoid activation function.

Implementation Differences

The difference between the three optimization methods lies in how the gradient of L is computed.

Full Gradient Descent Full GD computes the gradient using the entire dataset at once:

$$\nabla L = \frac{1}{n}X^T(y-\hat{y})$$

This results in one weight update per epoch. The gradient direction is stable but may converge slowly.

Stochastic Gradient Descent SGD updates the weights after processing each individual sample:

$$\nabla L^{(i)} = x^{(i)}(y^{(i)} - \hat{y}^{(i)})$$

This results in many updates per epoch (equal to the number of samples). The updates are noisy but often lead to faster convergence in terms of loss reduction.

Mini-Batch Gradient Descent Mini-batch GD computes the gradient over small subsets of size b:

$$\nabla L_{batch} = \frac{1}{b} X_{batch}^T (y_{batch} - \hat{y}_{batch})$$

This approach balances stability and computational efficiency.

Loss Convergence Comparison

Figure 4 shows the convergence of L over epochs for the three methods.

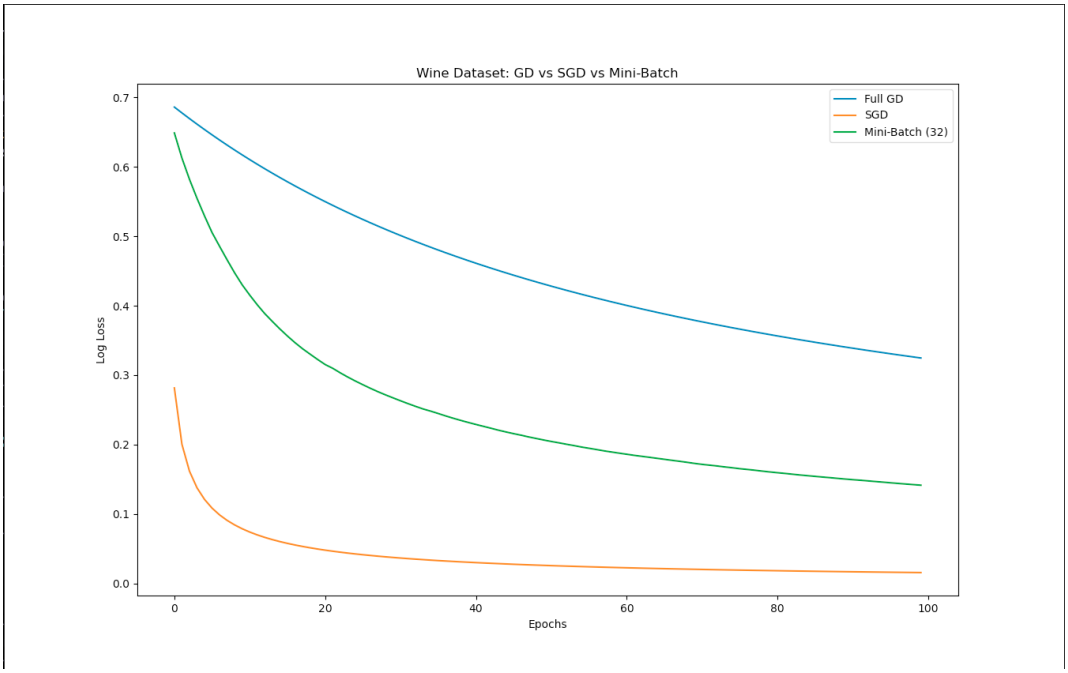

Figure 4: Loss convergence comparison on the Wine dataset.

From the figure, I observe:

- SGD reduces the loss most rapidly.
- Mini-batch SGD converges faster than full GD.
- Full GD shows the slowest decrease in loss.

Time Cost Comparison

The measured training times were:

- Full GD: 0.0033 seconds
- SGD: 0.1389 seconds
- Mini-batch SGD: 0.0073 seconds

Although SGD achieves faster loss reduction per epoch, it performs significantly more weight updates per epoch.

For the Wine dataset (approximately 130 samples):

- Full GD performs 1 update per epoch.
- Mini-batch (batch size 32) performs approximately 4 updates per epoch.
- SGD performs approximately 130 updates per epoch.

Over 100 epochs, SGD performs roughly 13,000 updates, whereas full GD performs only 100 updates. This explains the increased computational time of SGD.

Conclusion

I conclude that:

- Full GD is computationally efficient for small datasets but converges slowly.
- SGD converges fastest in terms of loss but has higher computational cost.
- Mini-batch SGD provides a balanced trade-off between convergence speed and time cost.